Cards API overview

When the GameWrapper is initialized, you can access the RGS features through `GameWrapper.rgs`.

The interface and the available methods depend on the `gameType` set during initialization. This document describes the methods and their responses for *'card'* games.

Note: To avoid blocked requests, it is recommended to check `GameWrapper.state` before calling RGS methods and only call them if the state is `active`. If the state is `paused`, the client should wait for a `game-resume` event before calling the desired method.

Common interfaces

RgsAccountData

```
{
  balance: number,
  currency: string
}
```

RgsCommonResponse

```
{
  config: GameClientConfig | null,
  context: RgsContextData,
  game: RgsGameData,
  spin?: RgsCardsSpinData,
  freespin?: RgsCardsSpinData,
  m?: any                              // metamorphic
  j?: any                              // jackpot
}
```

GameClientConfig

Everything that is set in the game JSON's field `ClientConfiguration`.

```
// example
{
  stakeValues?: number[];
}
```

RgsContextData

```
{
  data: {
```

```
    currency: string,
    currentBalance: number
  },
  playerId: string
}
```

RgsGameData

```
{
  nextAction: string,
  action: string,
  totalWin?: number,
  totalBet?: number,
  ticketId?: string,
  name?: string
}
```

RgsCardsSpinData

```
{
  pH: CardsHand[],      // Player hands
  tW: number,           // Total win
  exec?: CardsExec[],   // Executed commands
  tB?: number,          // Total bet
  sA?: number,          // Free spins awarded
  sR?: number,          // Free spins remaning
  sRt?: number,         // Free spin retrigger
  fsC?: any,            // Free spin context
}

// CardsExec
{
  cmd: string,  // Command
  cnt: int,     // Count
  utc: string   // Time
}

// CardsHand
{
  s?: number[],     // Player selected cards
  h?: number[],     // List of dealt cards (hand)
  bT?: number,      // Bet type
  d?: number[]      // Deck
  tB?: number,      // Total bet
  wA?: number,      // Win amount of the spin
  w?: number[{}],   // Individual winnings on cards
  f?: any,          // Feature
```

```
  sA?: number,      // Free spins awarded
  sR?: number,      // Free spins remaining
}
```

Methods

- `getAccount()`
- `refresh()`
- `spin(bet: number, selectedCards: number[][])`
- `freeSpin(selectedCards: number[][])`
- `execute(commands: string[][])`
- `getPreviousTickets()`
- `close()`
- `getTicketData(ticketId?: string)`
- `setTicketData(data: string | object, ticketId?: string)`
- `getTicket(ticketId: string)`
- `sendCustomRequest(service: string, method: string, args: any[])`

`getAccount()`

Get user's current balance data including the currency. Calling this method will trigger a balance update throughout the whole GameWrapper.

Return value

- `Promise<RgsAccountData>` - a promise that resolves to `RgsAccountData` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getAccount()
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Example response

```
{
  balance: 10000,
  currency: "GBP"
}
```

`refresh()`

Get game configuration data and the next pending action (*spin* or *close*). `refresh()` should be the first RGS API call after GameWrapper is initialized.

Usage example

```
wrapper.rgs.refresh()
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Example responses

```
{
  config: null,
  context: {...},
  game: {
    nextAction: "spin",
    action: "refresh",
  },
  m: {...}
}
```

Example response in case of an unfinished game

```
{
  config: null,
  context: {...},
  game: {
        nextAction: "close",
        action: "refresh",
        totalBet: 1,
        ticketId: "4d8c133f-1a53-11ee-9927-000c2932a097",
        name: "piqum-classic"
  },
  m: {...},
  spin: {
    pH: [{...}],
    tB: 1,
```

```
    tW: 0
  }
}
```

`getGame()`

Get game data including stake, prize level and configuration data. `getGame()`.

Usage example

```
wrapper.rgs.getGame()
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

`spin(bet: number, selectedCards: number[][])`

Purchase and start a new spin.

Arguments

- `bet` - `{number}` bet amount for this spin.
- `selectedCards` - `{number[][]}` - Should be a one-dimensional array in the case of single-hand games, where array represents all cards on the screen, where 1 indicates a selected card and 0 stands for a non-selected card.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.spin(1, [[0, 1, 0, 0, 0]])
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Example response

Example response:

```
{
  config: null,
```

```
  context: {...},
  game: {
    nextAction: "close",
    action: "spin",
    totalBet: 1,
    ticketId: "eec91816-1a4d-11ee-a4a3-000c2932a097",
    name: "piqum-classic"
  },
  m: {...},
  spin: {
    pH: [
      h: [7, 0, 49, 34, 57],
      s: [1, 0, 0, 0, 0],
      tB: 1,
      f: [{
        id: "piqumplay",
        d: {...}
      }]
    ],
    tB: 1,
    tW: 0
  }
}
```

`freeSpin(selectedCards: number[][])`

Perform a free spin.

Arguments

- `selectedCards` - `{number[][]}` - Should be a one-dimensional array in the case of single-hand games, where array represents all cards on the screen, where 1 indicates a selected card and 0 stands for a non-selected card.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.freeSpin([[0, 1, 0, 0, 0]])
  .then((data) => {
    // process data recevied from RGS
```

```
  })
  .catch(handleErrors);
```

Example response

Example response:

```
{
  config: null,
  context: {...},
  game: {
    nextAction: "freespin",
    action: "freespin",
    totalBet: 1,
    ticketId: "eec91816-1a4d-11ee-a4a3-000c2932a097",
    name: "piqum-classic"
  },
  m: {...},
  freespin: {
    pH: [{...}],
    tB: 1,
    tW: 14,
    sA: 9,
    sR: 3,
  }
}
```

`execute(commands: string[][])`

Execute commands from argument's list.

Arguments

- `commands` - `{string[]}` - Array of commands.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.execute(["piqum-bonus:CancelBonus"])
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Example response

```
{
  config: null,
  context: {...},
  game: {
    action: "execute",
    name: "piqum-bonus-96",
    nextAction: "close",
    ticketId: "fbbdbd38-7a8a-11ef-9b53-46ecbde7559e",
    totalBet: 1
  },
  m: {
    exec: {
      cmd: 'piqum-bonus:CancelBonus',
      cnt: 1,
      utc: '2024-09-24T15:41:04.909Z'
    },
    hand: [...],
    meta: {},
    stat: { b:0, c: 0, w: 0}
  },
  spin : {
    pH: [{...}]
    tB: 1,
    tW: 0,
  }
}
```

`GetPreviousTickets()`

Returns the last 10 tickets of the specified game within a 24-hour interval. The elements of the array in the response are formatted same as in GetTicket response.

Return value

- `Promise<RgsCommonResponse[]>` - a promise that resolves to `RgsCommonResponse[]` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.GetPreviousTickets()
  .then((data) => {
    // process data recevied from RGS
```

```
  })
  .catch(handleErrors);
```

Example response

```
[
  {context: {...}, game: {...}, config: null, spin: {...}},
  {context: {...}, game: {...}, config: null, spin: {...}},
  {context: {...}, game: {...}, config: null, spin: {...}},
  ...
]
```

`close()`

Finish the current game and close the ticket.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.close()
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Example response

```
{
  config: null,
  context: {...},
  game: {
    action: "close",
    name: "piqum-classic",
    nextAction: "spin",
    ticketId: "f2c57fc5-1a4d-11ee-a4a3-000c2932a097",
    totalBet: 1
  }
}
```

`getTicketData(ticketId?: string)`

Get data that was associated with a certain ticket.

If `ticketId` isn't specified, the wrapper will get data associated with the current ticket.

Argument

- `ticketId` - `{string}` ticket ID for which to get data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was previously set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicketData('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Example response

```
'{"color": "red"}'
```

`setTicketData(data: string | object, ticketId?: string)`

Associate data with a ticket.

If `ticketId` isn't set it will default to the current ticket ID.

Arguments

- `data` - `{string | object}` the data to be associated with a ticket. May either be a JSON string or a serializable javascript object. The object will be stringified before sending the request to RGS.
- `ticketId` - `{string}` ticket ID for which to set data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

`getTicket(ticketId: string)`

Get ticket data for a previously played game. This is useful for getting historical ticket data, though this is usually handled by the GameWrapper and there is no need for the client to call this method. For more details about history replay, see the Usage Guide.

Argument

- `ticketId` - {string} - ticket ID of a previously played game

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicket('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```

Example response

```
{
  config: null,
  context: {...},
  game: {
    nextAction: '',
    action: '',
    totalBet: 12.5,
    ticketId: '0098e1ae-4a33-11e9-8225-acbc327d774f',
    name: 'piqum-classic'
  },
  m: {...},
  spin: {
    pH: [{...}],
    tB: 1,
    tW: 0
  }
}
```

`sendCustomRequest(service: string, method: string, args: any[])`

Send a custom RGS request through the wrapper. This is useful when the client wants to use a feature which is supported by the RGS, but not implemented by the wrapper.

Should be used with caution. Best course of action is to ask first if this is the right way to implement required functionality. That is to avoid unnecessary debugging and delays.

Argument

- `service` - `{string}` RGS service name
- `method` - `{string}` RGS method name
- `args` - `{any[]}` array of arguments for the method. May be an empty array for no arguments.

Return value

- `Promise<any>` - a promise that resolves to the response object once a valid response is received. The exact response structure depends on the RGS method that is being called. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
const gameId = /* gameId */;
wrapper.rgs.sendCustomRequest('CardService', 'Refresh', [gameId])
  .then((data) => {
    // process data recevied from RGS
  })
  .catch(handleErrors);
```